

提高组 CSP-S 2024 初赛模拟卷 7

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 以下哪个操作系统版本不属于 Linux 操作系统？（ ）
A. Ubuntu B. CentOS C. RedHat D. iOS
2. 对于已经排好序的数列，以下排序算法最快的是（ ）。
A. 插入排序 B. 基数排序 C. 选择排序 D. 快速排序
3. 以下哪个排序从顺序存储改为链式存储时间复杂度会增加？（ ）
A. 冒泡排序 B. 插入排序 C. 希尔排序 D. 堆排序
4. 一个有 54 个结点（根结点深度为 1）的完全四叉树，按前序遍历的顺序给结点从 1 开始编号，则第 30 号结点的父结点是第（ ）号。
A. 5 B. 6 C. 7 D. 8
5. 有 5 枚硬币，第 i 枚硬币有 $\frac{p_i}{q_i}$ 的概率抛出正面（剩余概率出现反面）。这些硬币抛完后，正面数量超过反面数量的概率是多少？假设 $p_1 = 3, q_1 = 5, p_2 = 1, q_2 = 7, p_3 = 1, q_3 = 1, p_4 = 0, q_4 = 1, p_5 = 4, q_5 = 6$ 。（ ）
A. $\frac{1}{35}$ B. $\frac{13}{35}$ C. $\frac{7}{15}$ D. $\frac{8}{21}$
6. 对二叉树 T 来说， $L(T)$ 表示 T 的根的左子树， $R(T)$ 表示 T 的根的右子树。二叉树 T 与二叉树 S 的大小关系定义如下。
 - 首先，结点数 5c11 的二叉树更小。
 - 其次，若结点数相同，则比较 $L(S)$ 与 $L(T)$ ，左子树更小的二叉树更小。
 - 若左子树也一样大，则比较 $R(S)$ 与 $R(T)$ ，右子树更小的二叉树更小。二叉树 T 的中序序列定义如下。
 - 先输出 $L(T)$ 的中序序列，并在外加一层括号，若 $L(T)$ 为空则跳过这步。
 - 再输出一个 $*$ 。
 - 最后输出 $R(T)$ 的中序序列，并在外加一层括号，若 $R(T)$ 为空则跳过这步。

将所有二叉树按照上述规则从小到大排序,第 10 小的二叉树的中序序列是什么?

()

- A. $((*)*(*))^*$ B. $(*)*((*)^*)$ C. $*(((*)^*)^*)$ D. $(*(*)^*)^*(*)$

7. 下述哪个关于欧拉图和哈密顿图的说法正确? ()

- A. 一张图不能同时称为欧拉图和哈密顿图
B. 欧拉图不一定含有欧拉回路
C. 只有有向图拥有欧拉回路
D. 无向连通图 G 是欧拉图,当且仅当 G 有零个、一个或两个奇数度的结点

8. 老师现在需要给 3 名学生分配 25 颗糖果,要求分给这 3 名学生分别不少于 1 颗、2 颗、3 颗糖果,问一共有多少种分法? ()

- A. 120 B. 150 C. 210 D. 231

9. 在双向链表中定位删除一个元素的平均复杂度为 ()。

- A. $O(1)$ B. $O(n)$ C. $O(\log n)$ D. $O(n \log n)$

10. G 是一个非连通简单无向图,共有 28 条边,则该图至少有 () 个顶点。

- A. 10 B. 9 C. 8 D. 7

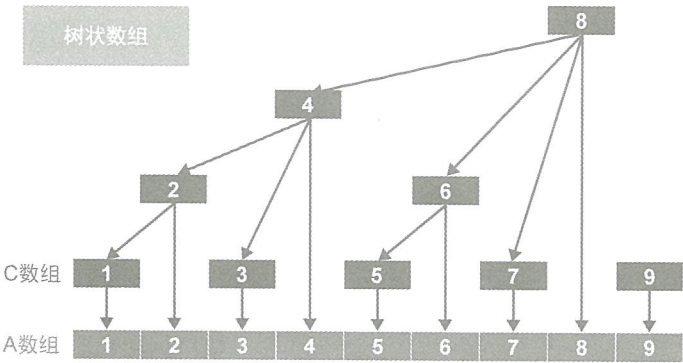
11. 堆排序建堆最快所需要的时间复杂度是 ()。

- A. $O(1)$ B. $O(n \log n)$ C. $O(\log n)$ D. $O(n)$

12. 学校里有 4 个盒子,第 1 个盒子有 3 朵花,第 2 个盒子有 3 朵花,第 3 个盒子有 3 朵花,第 4 个盒子有 6 朵花。同一个盒子内花的颜色相同,不同盒子内的花颜色不同。现要从中选出 4 朵花组成一束花,共有多少种方案?若两束花中相同颜色的花的数量都相同,则认为方案相同。()

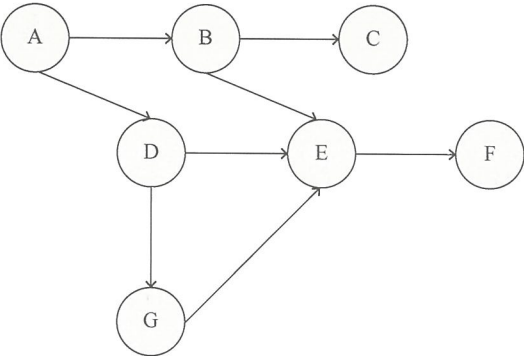
- A. 33 B. 25 C. 30 D. 27

13. 树状数组是解决区间问题常用的数据结构,其原理是使用 C 数组去求 A 数组的前缀和。如下图所示: $C[1] = A[1]$, $C[2] = C[1] + A[2]$, $C[4] = C[2] + C[3] + A[4]$ 。现定义概念组成值和被组成值,例如 $C[2] = C[1] + A[2]$,那么 $C[1]$ 、 $A[2]$ 都是 $C[2]$ 的组成值,反过来说 $C[2]$ 就是它俩的被组成值。可以发现,每一项 C 或者每一项 A 都最多成为别人的组成值一次。现请问 $C[111]$ 的被组成值是多少? ()



- A. 112 B. 113 C. 111 D. 127

14. 下图的拓扑序列结果有多少种？（ ）



- A. 1 B. 7 C. 6 D. 9

15. 现有 4 枚硬币，甲乙两人轮流给硬币未涂色的一面涂上颜色（红或者黑），当 8 次涂色完成后，所有硬币被全部抛出。如果红色面向上更多则甲胜，否则乙胜；如果两种颜色面向上的数量相等，则没有获胜方。如果甲乙两人都采取最优策略使得各自获胜的概率最大，则甲获胜的概率为（ ）。

- A. $\frac{5}{16}$ B. $\frac{3}{8}$ C. $\frac{3}{16}$ D. $\frac{1}{2}$

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特殊说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include<iostream>
02
```

```
03 using namespace std;
04
05 typedef long long ll;
06
07 const int N = 25, mod = 1e9 + 7;
08
09 ll a[N];
10
11 ll cal(ll a, ll b){
12     if(a < b) return 0;
13     ll x = 1;
14     for(ll i = a; i > a - b; i--){
15         x *= i;
16         x %= mod;
17     }
18     return x;
19 }
20
21 ll qmi(ll a, ll b){
22     ll s = 1;
23     while(b){
24         if(b & 1) s = s * a % mod;
25         a = a * a % mod;
26         b >>= 1;
27     }
28     return s;
29 }
30
31 int main(){
32     int n; ll m;
33     cin >> n >> m;
34     for(int i = 0; i < n; i++){
35         cin >> a[i];
36     }
37
38     ll s = 1;
39
40     for(int i = 1; i <= n - 1; i++){
41         s = s * i % mod;
42     }
43     s = qmi(s, mod - 2);
44 }
```

```
45    ll ans = 0;
46    for(int i = 0; i < (1 << n); i++){
47        ll h = n + m - 1; int sign = 1;
48        for(int j = 0; j < n; j++){
49            if(i & (1 << j)){
50                h -= a[j] + 1;
51                sign = ~sign + 1;
52            }
53        }
54        h = cal(h, n - 1);
55        ans += h * s * sign % mod;
56        ans %= mod;
57    }
58
59    cout << (ans % mod + mod) % mod << endl;
60
61    return 0;
62 }
```

■ 判断题

16. 将第 41 行改成 $s *= i \% \text{mod}$, 结果不会改变。 ()
17. 将第 51 行改成 $\text{sign} = 1 - \text{sign}$, 结果不会改变。 ()
18. 在不考虑 mod 的情况下, 当 n 固定时, m 增大, 最后输出的值一定增大。 ()
19. 该程序运用的思想是容斥思想。 ()

■ 选择题

20. 当输入 4 4 3 3 3 6 时, 程序输出 ()。
- A. 30 B. 31 C. 32 D. 33
21. 该程序的时间复杂度是 ()。
- A. $O(n \log n)$ B. $O(2^n n)$ C. $O(2^n \log n)$ D. $O(2^n (n + \log n))$

(2)

```
01 #include<iostream>
02 using namespace std;
03
04 int phi[40005];
05
```

```
06 int get(int n){
07     int sum = n;
08     for(int i = 2; i < n; i++){
09         if(n % i == 0){
10             sum -= sum / i;
11             while(n % i == 0) n /= i;
12         }
13     }
14     if(n > 1) sum -= sum / n;
15     return sum;
16 }
17
18 int main(){
19     int n;
20     scanf("%d",&n);
21     if(n == 1){
22         return 0;
23     }
24     for(int i = 2; i <= n; i++){
25         phi[i] = i;
26     }
27     for(int i = 2; i <= n; i++){
28         if(phi[i] != i) continue;
29         for(int j = i; j <= n; j += i)
30             phi[j] = phi[j] / i * (i-1);
31     }
32     int ans = 1;
33     for(int i = 1; i < n; i++) ans += phi[i];
34
35     int ans2 = 0;
36     for(int i = 2; i <= n - 1; i++) ans2 += get(i);
37
38
39     printf("%d %d",ans * 2 + 1,ans2 * 2 + 3);
40 }
```

■ 判断题

22. 求解 phi 数组的算法的时间复杂度为 $O(n\log n)$ 。 ()
23. 将第 8 行代码改成 `for(int i = 2; i * i <= n; i++)` 不会导致答案错误。 ()
24. (3 分) 最后输出的两个值总是相等。 ()

■ 选择题

25. 求解 ans2 值的算法的时间复杂度为 ()。
- A. $O(n^2)$ B. $O(n^2\sqrt{n})$ C. $O(n^2\log n)$ D. $O(n\log n)$
26. 若输入 20, 程序输出 ()。
- A. 241 241 B. 239 241 C. 255 257 D. 205 205
27. 当输入 35 时, phi[36]与 get(36)分别等于 ()。
- A. 12 12 B. 0 0 C. 11 12 D. 0 12

(3)

```
01 #include<iostream>
02 #include<cstring>
03 #include<algorithm>
04 using namespace std;
05 #define MOD 1000000007
06 char s[1200000];
07 int nt[1200000];
08 int jp[21][1200000];
09 int main() {
10     scanf("%s",s+1);
11     nt[1] = 0;
12     int l = strlen(s+1);
13     for(int i = 2; i <= l; ++i) {
14         int j = nt[i - 1];
15         while(j && s[i] != s[j + 1]) j = nt[j];
16         if(s[i] == s[j + 1]) nt[i] = j + 1;
17         else nt[i] = 0;
18         jp[0][i] = nt[i];
19     }
20     for(int j = 1; j < 20; ++j)
21         for(int i = 1; i <= l; ++i)
22             jp[j][i] = jp[j - 1][jp[j - 1][i]];
23     int ans = 1;
24     for(int i = 2; i <= l; ++i) {
25         int tt = i;
26         for(int j = 19; j >= 0; --j)
```

```

27         if(jp[j][tt] * 2 > i) tt = jp[j][tt];
28     int gg = 0;
29     for(int j = 19; j >= 0; --j)
30         if(jp[j][tt]) gg += 1 << j, tt = jp[j][tt];
31     ans = 1ll * ans * (gg + 1) % MOD;
32 }
33 printf("%d\n", ans);
34 return 0;
35 }

```

■ 判断题

28. (3 分) 当输入的字符串为 abcaabcaab 时, nt[10] 为 5。 ()
29. 当输入的字符串为 abcaabcaab 时, 第 15 行中的 $j = nt[j]$ 一共执行 1 次。 ()
30. 该程序最后的输出结果可能为 0。 ()

■ 选择题

31. 当输入为 aaaaaa 时, 程序的输出为 ()。
- A. 12 B. 24 C. 36 D. 48
32. (4 分) 若不考虑 % MOD, 字符串长度至少为 (), 程序的输出大于 100。
- A. 5 B. 6 C. 7 D. 8

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 一共有 n 个数, 第 i 个数 x_i 可以取 $[a_i, b_i]$ 中的任意值。设 $S = \sum x_i^2$, 求 S 的种类数。
 $1 \leq n, a_i, b_i \leq 100$ 。考虑极限情况下 S 最大为 1 000 000, 使用 DP 解题。题目时限 1s, 空间限制为 256 MB。

```

01 #include <bits/stdc++.h>
02 #define maxn 1000005
03 using namespace std;
04 bitset<maxn> bit1, bit2;
05 int a[105], b[105];
06 int dp[N][
07 int main() {
08     int n;
09     scanf("%d", &n);
10     for (int i = 1; i <= n; i++) {

```



```
11     scanf("%d%d", &a[i], &b[i]);
12 }
13 for (int i = 1; i <= n; i++) {
14     int l = a[i], r = b[i];
15     for (int j = l; j <= r; j++) {
16         if (i == 1)
17             ①;
18         else
19             ②;
20     }
21     if (i == 1)
22         bit2 = bit1;
23     bit1 = bit2;
24     ③;
25 }
26 printf("%d\n", ④);
27 }
```

33. ①处应填 ()。

A. bit1.set(j * j)

B. bit2.set(j * j)

C. bit1.set(i * i)

D. bit2.set(i * i)

34. ②处应填 ()。

A. bit2 ^= bit1 << (j * j)

B. bit2 += bit1 << (j * j)

C. bit2 |= bit1 << (j * j)

D. bit2 &= bit1 << (j * j)

35. ③处应填 ()。

A. bit1.reset()

B. bit2.reset()

C. bit2.set(0)

D. bit1.set(0)

36. ④处应填 ()。

A. bit1

B. bit2.count()

C. bit1.count()

D. ~bit1.count()

(2) 有 n 个人, 每个人的空闲时间为 $[a_i, b_i]$, 现需要将 n 个人分至 k 个组别中。每个组的成员能一起玩的时间是所有人空闲时间的交集。现希望知道怎么分组才能使得组成员一起玩的时间总和最大。需要注意的是, 每个组一起玩的时间至少要大于一分钟,

每个组的成员至少为 1 位。

解题思路：

将题目中的空闲时间区间分为两种——可以包含其他区间的大区间，以及小区间。对于包含其他区间的大区间，最优的情况即是单独一组或者与其被包含的区间一组。所以可以先将这些大区间去掉，观察剩余区间的性质。

```
01 #include<iostream>
02 #include<cstring>
03 #include<algorithm>
04 #include<map>
05 #include<vector>
06
07 using namespace std;
08
09 const int N = 5e3 + 10;
10 int n,k;
11 struct node{
12     int l,r;
13     bool operator < (const node &p)const{
14         if(l == p.l) return r > p.r;
15         return l < p.l;
16     }
17 }a[N],b[N];
18
19 int seg[N];
20
21 int dp[N][N]; //前 i 个数用了 j 个片段
22
23 int q[N][N],tail[N],head[N];
24
25 int main(){
26     cin >> n >> k;
27     for(int i = 1; i <= n; i++){
28         cin >> a[i].l >> a[i].r;
29     }
30
31     sort(a + 1,a + n + 1);
32
33     int cnt = 0,tot = 1;
34     int maxr = 0x3f3f3f3f;
35     for(int i = n; i >= 1; i--){
```

```

36         if(maxr <= a[i].r){
37             seg[tot++] = a[i].r - a[i].l;
38         }else{
39             b[++cnt] = a[i];
40             ①;
41         }
42     }
43     memset(dp,-0x3f,sizeof dp);
44     sort(b + 1,b + cnt + 1);
45     ②;
46
47     dp[0][0] = 0;
48     for(int i = 1; i <= k; i++) tail[i] = -1;
49     for(int i = 1; i <= cnt; i++){
50         for(int l = k; l >= 1; l--){
51             while(dp[q[l][tail[l]] - 1][l - 1] + b[q[l][tail[l]]].
                    r < ③ && tail[l] >= head[l]) --tail[l];
52             q[l][++tail[l]] = i;
53             while( ④ && head[l] <= tail[l]){
54                 ++head[l];
55             }
56             dp[i][l] = max(dp[i][l],dp[q[l][head[l]] - 1][l - 1] + ⑤);
57         }
58     }
59
60     int sum = 0;
61     int maxn = dp[cnt][k];
62     for(int i = 1; i <= tot && i <= k; i++){
63         sum += seg[i];
64         maxn = max(maxn, ⑥);
65     }
66
67     if(maxn > 0) cout << maxn << endl;
68     else cout << 0 << endl;
69
70
71     return 0;
72 }

```

37. ①处应填 ()。

A. maxr = a[i].l

B. seg[++tot] = maxr - a[i].l

D. --tot

```
A. sort(seg + 1, seg + tot, greater<int>())
```

```
C.sort(seg + 1,seg + tot + 1,greater<int>())
```

```
D.sort(seg,seg + tot,less<int>())
```

A. $dp[i - 1][l - 1] + b[i].r$

```
C.dp[i - 1][l - 1] + b[i - 1].r
```

$$D. dp[i - 1][l - 1] + b[i - 1].l$$

```
A. b[q[l - 1][head[l - 1]]].r <= b[i].l
```

```
B. b[q[l - 1][tail[l - 1]]].r <= b[i].l
```

```
C.b[q[l][tail[l]]].r <= b[i].l
```

D. `b[q[l][head[l]]].r <= b[i].l`

A. $b[q[l][head[l]]].l - b[i].l$

```
B. b[q[l - 1][head[l - 1]]].r - b[i - 1].l
```

```
C.b[q[l - 1][head[l - 1]]].r - b[i].l
```

$$D.b[q[l][head[l]]].r - b[i].l$$

```
A. sum + dp[cnt][k]
```

```
B.sum + dp[cnt][k - i]
```

```
C.sum + dp[k - i][cnt]
```

D. sum + dp[i][cnt]